

Web Engineering Lab CS-352

Week 7

Lab Designed By: Abdul Rehman Qureshi

Lab Objectives

By the end of this lab, students will be able to:

- Explain what the Bootstrap framework is, the problem it solves, and why it became the industry standard for rapid, responsive front-end development.
- Set up Bootstrap in an HTML project using both the CDN method and the downloaded source-file method, and explain the trade-offs of each.
- Understand Bootstrap's mobile-first responsive design philosophy and apply the 12-column grid system with its breakpoints (sm, md, lg, xl, xxl).
- Use core Bootstrap typography, color, and spacing utility classes to style text and layout without writing custom CSS.
- Implement core UI components: alerts, badges, breadcrumbs, buttons and button groups, cards, carousels, collapsibles/accordions, dropdowns, forms, input groups, list groups, navs, and navbars.
- Implement interactive components that depend on Bootstrap's JavaScript bundle: modals, toasts, popovers, and tooltips, and explain why the JS bundle must be linked for these to work.
- Build paginators, progress bars, spinners, and responsive tables using Bootstrap's built-in classes.
- Apply Bootstrap's styling utilities for floats, Flexbox alignment, borders, shadows, position, and visibility to fine-tune layouts.
- Combine multiple Bootstrap components into a single, coherent, responsive web page.

Tools / Software Required

- Visual Studio Code (or any code editor)
- Google Chrome (with DevTools) for rendering, inspecting, and resizing the viewport
- Internet connection (to load Bootstrap via CDN) or a downloaded copy of the Bootstrap source files for offline use
- Live Server extension (optional, for auto-reloading the browser on save)
- A modern web browser to cross-check rendering (Firefox/Edge)

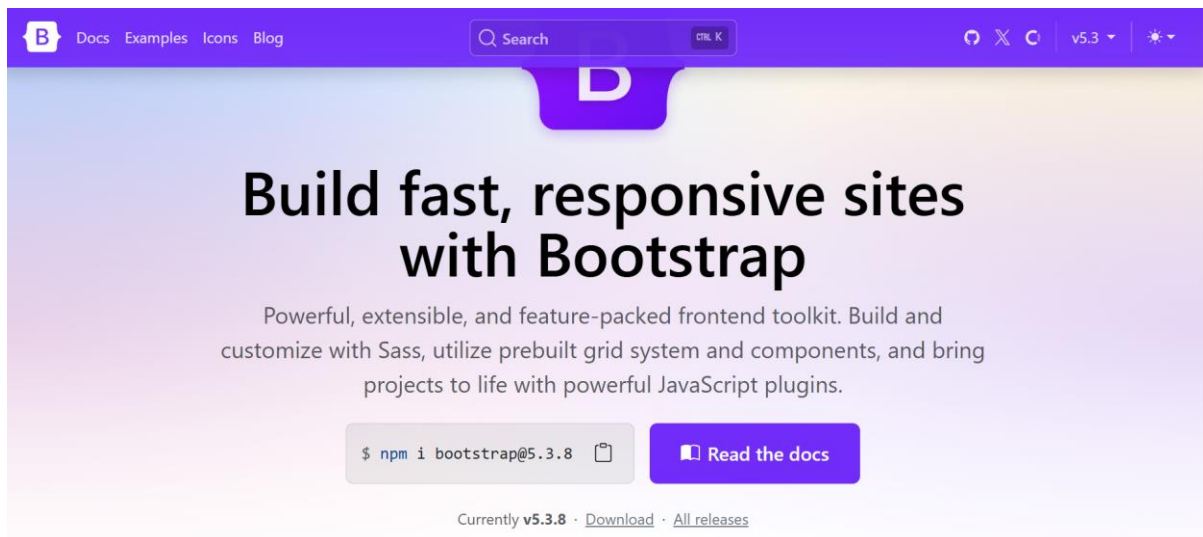
Lab Overview

In the previous lab, you learned to write CSS by hand — selectors, the box model, colors, and units. Writing every rule from scratch works, but it is slow, and every project ends up reinventing the same wheels: a responsive grid, a styled button, a modal dialog, a navigation bar that collapses on mobile.

Bootstrap is a free, open-source CSS and JavaScript framework, originally built at Twitter (released in 2011 as 'Twitter Blueprint'), that packages these common UI patterns into ready-made, pre-tested classes and components. Instead of writing `.btn { padding: ...; border-radius: ...; }` yourself, you simply add `class="btn btn-primary"` to a `<button>` and Bootstrap's shipped CSS does the rest.

Bootstrap is built on a mobile-first philosophy: styles are written for small screens first, then progressively enhanced for larger screens using min-width media queries. This is combined with a 12-column responsive grid system, so the same HTML markup automatically rearranges itself from a stacked single column on a phone to a multi-column layout on a desktop, without you writing a single `@media` query. This is the core productivity gain that made Bootstrap the most widely used CSS framework in the world for over a decade.

Real-World Anchor: Government and enterprise portals — including many university LMS platforms, internal dashboards, and early versions of products like Twitter's own settings pages — are frequently built on Bootstrap because it lets small teams ship consistent, accessible, responsive interfaces quickly without hiring a dedicated design system team. Even today, when a developer needs a professional-looking internal tool by tomorrow morning, Bootstrap remains a default first choice.



Task 1 — Introduction to Bootstrap

1.1 Concept

Bootstrap ships two things: a CSS file (`bootstrap.min.css`) containing thousands of pre-written utility and component classes, and a JavaScript file (`bootstrap.bundle.min.js`) that powers interactive components such as modals, dropdowns, carousels, and tooltips. The CSS alone is enough for static layout and styling; the JavaScript bundle (which includes Popper.js for positioning) is required the moment a component needs to open, close, slide, or reposition itself in response to user interaction.

Getting started

Introduction

[Download](#)[Contents](#)[Browsers & devices](#)[JavaScript](#)[Webpack](#)[Parcel](#)[Vite](#)[Accessibility](#)[RFS](#)[RTL](#)[Contribute](#)

Customize

[Overview](#) [Sass](#)

Get started with Bootstrap

[View on GitHub](#)

Bootstrap is a powerful, feature-packed frontend toolkit. Build anything—from prototype to production—in minutes.

Quick start

Get started by including Bootstrap's production-ready CSS and JavaScript via CDN without the need for any build steps. See it in practice with this [Bootstrap CodePen demo](#).

1. Create a new `index.html` file in your project root. Include the `<meta name="viewport">` tag as well for [proper responsive behavior](#) in mobile devices.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-sca
    <title>Bootstrap demo</title>
  </head>
  <body>
    <h1>Hello, world!</h1>
  </body>
</html>
```

2. Include Bootstrap's CSS and JS. Place the `<link>` tag in the `<head>` for our CSS, and the `<script>` tag for our JavaScript bundle (including Popper for positioning dropdowns, popovers, and tooltips) before the closing `</body>`. Learn more about our [CDN links](#).

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-sca
    <title>Bootstrap demo</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/
  </head>
  <body>
    <h1>Hello, world!</h1>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist
  </body>
</html>
```

You can also include [Popper](#) and our JS separately. If you don't plan to use dropdowns, popovers, or tooltips, save some kilobytes by not including Popper.

```
<script src="https://cdn.jsdelivr.net/npm/@popperjs/core@2.11.8/di
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/
```

On this page

Quick start

[CDN links](#)[Next steps](#)[JS components](#)[Important globals](#)[HTML5 doctype](#)[Viewport meta](#)[Box-sizing](#)[Reboot](#)[Community](#)

Check out the latest remote job listings from

Key Elements of Bootstrap

- Grid System — a 12-column, responsive, mobile-first layout engine built with Flexbox.
- Reboot — a modern, cross-browser CSS reset baked into Bootstrap so elements render consistently in every browser.
- Utility Classes — small, single-purpose classes (m-3, text-center, d-flex, fw-bold) that apply one CSS rule each, avoiding the need to write custom CSS for common tweaks.
- Components — pre-built UI patterns (navbars, cards, modals, carousels) with baked-in accessibility (ARIA attributes) and cross-browser behavior.
- JavaScript Plugins — bundled, dependency-free (as of Bootstrap 5) JS behaviors for interactive components.
- Theming — Sass variables and CSS custom properties that let you re-brand the entire framework (colors, fonts, spacing scale) from one place.

Source / Setup Methods

There are two standard ways to bring Bootstrap into a project. The CDN method is fastest for labs and prototypes: you link Bootstrap's CSS and JS directly from a public Content Delivery Network, so there is nothing to download and the files are often already cached in the visitor's browser from another site. The downloaded/local method copies the Bootstrap source files into your own project folder — preferred for production apps that must work offline, behind strict corporate firewalls, or where every asset must be self-hosted for security and licensing audits.

```
<!-- CDN Method (recommended for this lab) -->
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Bootstrap Page</title>
  <!-- Bootstrap CSS -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <!-- page content goes here -->

  <!-- Bootstrap JS Bundle (includes Popper) — placed just before </body> -->
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
</script>
</body>
```

Notice the viewport `<meta>` tag. Bootstrap's responsive breakpoints are driven by CSS media queries that read the device's reported width; without this tag, mobile browsers report a fake 'desktop' width and Bootstrap's mobile-first styles never activate. This single line is the most commonly forgotten — and most consequential — line in any Bootstrap page.

Screen Sizes / Breakpoints

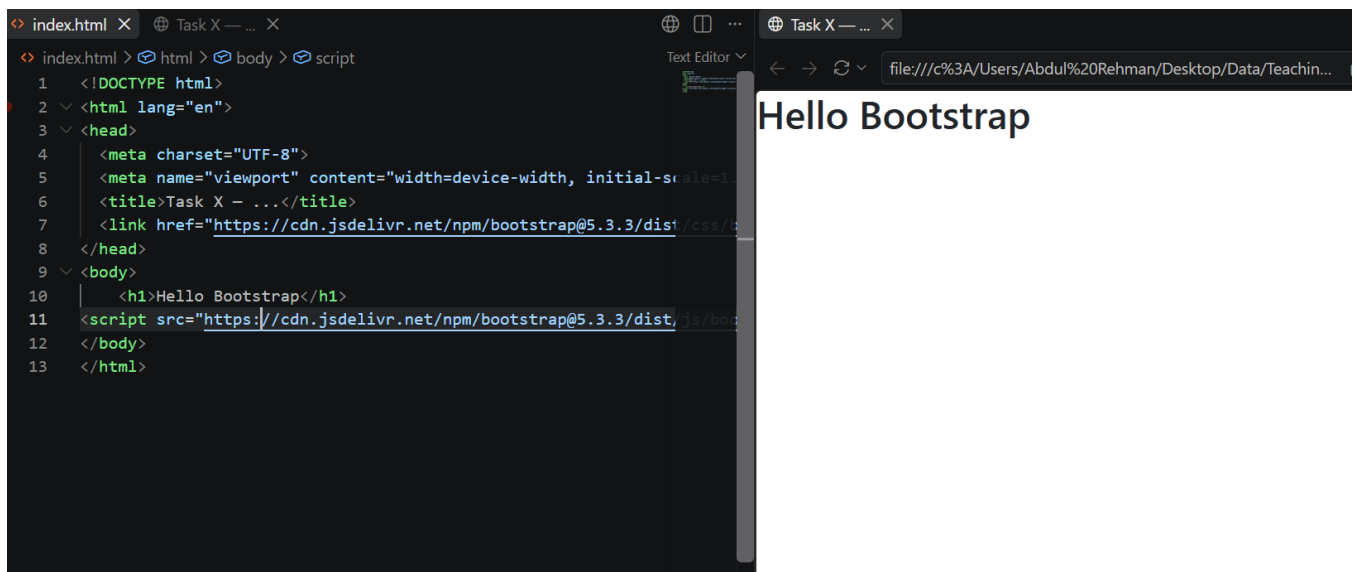
Bootstrap defines six breakpoints. A class suffixed with a breakpoint abbreviation (e.g. `col-md-6`) applies from that breakpoint's minimum width upward, which is what makes the system mobile-first: unprefixed classes (`col-6`) apply to all sizes, and each larger breakpoint can override it.

Breakpoint	Class Infix	Min Width	Typical Device
Extra small	(none)	<576px	Phones (portrait)

Breakpoint	Class Infix	Min Width	Typical Device
Small	sm	≥576px	Phones (landscape)
Medium	md	≥768px	Tablets
Large	lg	≥992px	Laptops
Extra large	xl	≥1200px	Desktops
Extra extra large	xxl	≥1400px	Large desktops / wide monitors

1.2 Lab Task

- Create a folder named bootstrap-lab. Inside it, create index.html.
- Add the Bootstrap 5 CSS CDN link inside <head> and the Bootstrap JS bundle CDN script just before </body>.
- Add the responsive viewport <meta> tag and confirm (via View Source) that both Bootstrap files load without a 404 error in the DevTools Network tab.
- Add a single <h1> with the text 'Hello Bootstrap' and observe, using DevTools, that the font-family and margins have already changed from the browser default — this is Bootstrap's Reboot at work before you have written any class of your own.
- Resize the Chrome window from full width down to a phone width and note in your report at which pixel width the layout would begin adapting once you add grid classes in the next task.



Task 2 — The Bootstrap Grid System & Breakpoints

2.1 Concept

Bootstrap's layout engine is built on three nested levels: a `.container` (or `.container-fluid`) that centers and constrains content width, a `.row` inside it that acts as a Flexbox row and cancels out unwanted edge spacing, and one or more `.col-*` elements inside the row that become Flexbox items. The 12-column system means you allocate columns out of a total of 12: three equal columns are each `col-md-4` ($4 + 4 + 4 = 12$), while a sidebar-plus-content layout might be `col-md-3` and `col-md-9`.

The difference between `.container` and `.container-fluid` mirrors a very common real design decision: `.container` has a maximum width at each breakpoint (so text lines do not stretch uncomfortably wide on

an ultra-wide monitor), while `.container-fluid` always spans 100% of the viewport width, which is preferred for dashboards, hero banners, and image galleries that should fill the screen edge-to-edge.

Real-World Anchor: News and e-commerce sites (e.g. Daraz, BBC) commonly use a bounded `.container` for article/product text so lines of text stay a comfortable reading width, but switch to a full-bleed `.container-fluid` for hero image banners at the top of the page — exactly the pattern you will build in this task.

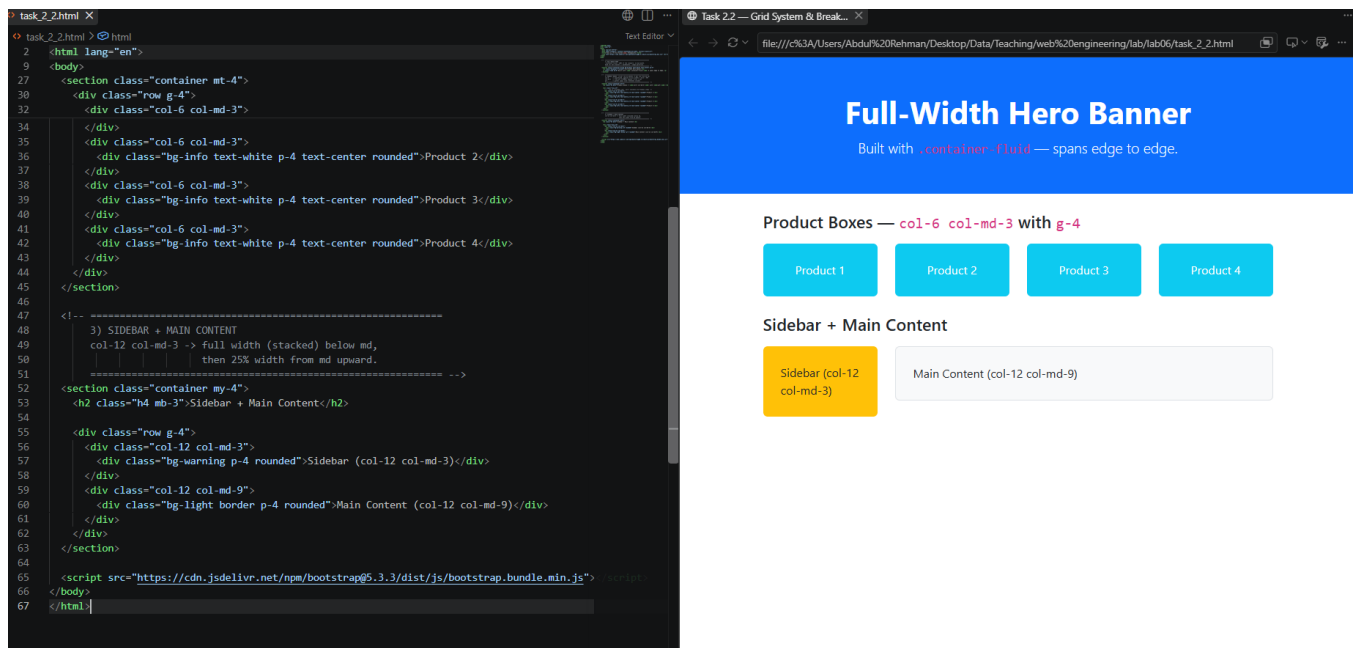
```
<div class="container">
  <div class="row">
    <div class="col-md-4">Column 1</div>
    <div class="col-md-4">Column 2</div>
    <div class="col-md-4">Column 3</div>
  </div>
</div>

<!-- Responsive: stacked on phones, sidebar layout from md upward -->
<div class="container">
  <div class="row">
    <div class="col-12 col-md-3">Sidebar</div>
    <div class="col-12 col-md-9">Main Content</div>
  </div>
</div>
```

Combining infixes lets one element behave differently at different sizes. `class="col-12 col-md-3"` means: take the full 12 columns (100% width, stacked) below the md breakpoint, then take only 3 of 12 columns (25% width, side-by-side) from md upward. Gutters — the small gap between columns — are automatic; you can widen or remove them with `g-*` utility classes (e.g. `g-4`, `g-0`).

2.2 Lab Task

- Inside a `.container`, build a row of 4 equal-width product boxes using `col-6 col-md-3`, so phones show 2 per row and desktops show 4 per row.
- Below that, build a sidebar + main-content layout using `col-12 col-md-3` for the sidebar and `col-12 col-md-9` for the content, and add a background color to each so the split is visible.
- Add a full-width `.container-fluid` hero section above both rows with a background color and centered heading text.
- Apply a gutter utility class (e.g. `g-4`) to increase spacing between the 4 product boxes and observe the change in DevTools.
- Resize the browser window slowly from desktop width to phone width and confirm the columns reflow at exactly the md breakpoint (768px), not gradually.



Task 3 — Typography, Colors & Alignment Utilities

3.1 Concept

Bootstrap replaces most hand-written CSS for text and spacing with single-purpose utility classes. Instead of writing a custom rule to bold and center a heading, you write `class="fw-bold text-center"`. This 'utility-first' approach for common tweaks (distinct from a full utility framework like Tailwind, but philosophically similar for these small cases) keeps your CSS file small because you reuse the same handful of classes across the whole site instead of inventing a new class per element.

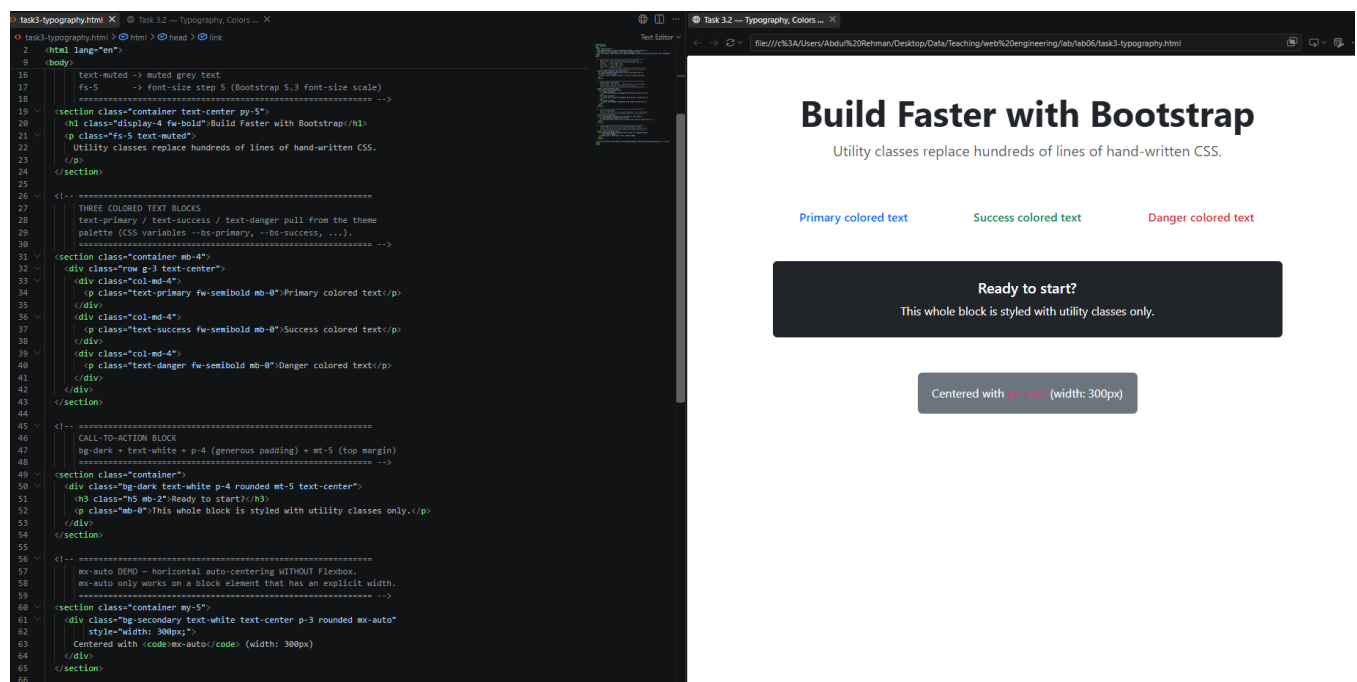
Category	Example Classes	Effect
Text alignment	<code>text-start</code> / <code>text-center</code> / <code>text-end</code>	Aligns text left / center / right
Font weight	<code>fw-bold</code> / <code>fw-normal</code> / <code>fw-light</code>	Sets font-weight
Font style	<code>fst-italic</code> / <code>fst-normal</code>	Sets font-style
Text color	<code>text-primary</code> / <code>text-danger</code> / <code>text-muted</code>	Sets text color from the theme palette
Background color	<code>bg-primary</code> / <code>bg-success</code> / <code>bg-dark</code>	Sets background-color from the theme palette
Spacing (margin)	<code>m-3</code> , <code>mt-2</code> , <code>mb-4</code> , <code>mx-auto</code>	Sets margin on all sides, top, bottom, or auto horizontal centering
Spacing (padding)	<code>p-3</code> , <code>pt-2</code> , <code>px-4</code>	Sets padding on all sides, top, or left+right
Display	<code>d-none</code> , <code>d-block</code> , <code>d-flex</code>	Sets the CSS display property

Bootstrap's spacing utilities follow the pattern `{property}{sides}-{size}`: the property is `m` (margin) or `p` (padding); sides is `t/b/s/e/x/y` (top, bottom, start, end, horizontal, vertical) or omitted for all sides; size is 0–5 (a fixed spacing scale) or `auto`. So `mt-4 mb-2 mx-auto` means 'margin-top: size 4, margin-bottom: size 2, and center this element horizontally.' Memorizing this pattern lets you guess almost any spacing class correctly without looking it up.

Real-World Anchor: Marketing landing pages (SaaS pricing pages, product launch pages) rely heavily on text-center, fw-bold, and the bg-* / text-* color utilities to quickly produce a polished, consistent 'hero section + centered heading + colored call-to-action' pattern without a designer writing custom CSS for every campaign page.

3.2 Lab Task

- Create a hero section with a large, bold, centered heading using display-4 fw-bold text-center.
- Add a subtitle paragraph below it styled with text-muted and fs-5.
- Create three short text blocks colored using text-primary, text-success, and text-danger respectively.
- Add a call-to-action button-like <div> with bg-dark text-white and generous padding (p-4) and a top margin (mt-5).
- Use mx-auto on a fixed-width box to demonstrate horizontal auto-centering without flexbox.



Task 4 — Alerts, Badges & Breadcrumbs

4.1 Concept

Alerts, badges, and breadcrumbs are small feedback and orientation components that appear on almost every production site. An alert communicates the status of an action (success, warning, error, info) using consistent color coding so users learn the meaning once and recognize it everywhere. A badge is a small, pill-shaped label used to show a count or status next to other content (a cart item count, an 'unread' indicator, a status tag). A breadcrumb shows the user's current location within a site's hierarchy and offers one-click navigation back up that hierarchy.

Real-World Anchor: An e-commerce checkout page shows a green success alert after 'Order Placed', a red badge with a number on the cart icon showing item count, and a breadcrumb like Home > Electronics > Mobile Phones > iPhone 15 so the shopper can jump back to the category page in one click instead of using the browser back button repeatedly.

```
<!-- Alerts -->
<div class="alert alert-success" role="alert">Order placed successfully!</div>
```

```

<div class="alert alert-danger" role="alert">Payment failed. Please try
again.</div>

<!-- Dismissible alert (requires the JS bundle) -->
<div class="alert alert-warning alert-dismissible fade show" role="alert">
  Your session will expire soon.
  <button type="button" class="btn-close" data-bs-dismiss="alert" aria-
label="Close"></button>
</div>

<!-- Badges -->
<button class="btn btn-primary">Inbox <span class="badge bg-
danger">4</span></button>
<span class="badge bg-secondary">New</span>

<!-- Breadcrumb -->
<nav aria-label="breadcrumb">
  <ol class="breadcrumb">
    <li class="breadcrumb-item"><a href="#">Home</a></li>
    <li class="breadcrumb-item"><a href="#">Electronics</a></li>
    <li class="breadcrumb-item active" aria-current="page">Mobile Phones</li>
  </ol>
</nav>

```

The `data-bs-dismiss="alert"` attribute is a Bootstrap JavaScript data attribute — it tells the bundled JS which behavior to attach to that button without you writing any `addEventListener` code yourself. This 'data attribute API' pattern (prefixed `data-bs-` in Bootstrap 5) is used across nearly every interactive Bootstrap component, which is why the JS bundle script tag is mandatory for anything beyond static alerts.

4.2 Lab Task

- Create four static alerts, one each for `alert-success`, `alert-danger`, `alert-warning`, and `alert-info`, each with a meaningful message.
- Make one of the alerts dismissible using `alert-dismissible fade show` plus the close button, and confirm clicking the `×` removes it from the page.
- Add a badge showing a count on a button (e.g. a mock 'Notifications' button with a red badge showing '3').
- Build a 3-level breadcrumb trail representing a page deep inside a hypothetical website of your choice (e.g. a university site: Home > Departments > Computer Science).

```

task4-alerts-badges-breadcrumb.html > @html > @body-p-4
2 <html lang="en">
3 <body class="p-4">
4   <h2 class="h4 mb-3">1. Static Alerts (four contextual types)</h2>
5   <!-- role="alert" makes screen readers announce the message. -->
6   <div class="alert alert-success" role="alert">
7     <strong>Success!</strong> Your order has been placed.
8   </div>
9   <div class="alert alert-danger" role="alert">
10    <strong>Error!</strong> Payment failed. Please try again.
11  </div>
12  <div class="alert alert-warning" role="alert">
13    <strong>Warning!</strong> Your subscription expires in 3 days.
14  </div>
15  <div class="alert alert-info" role="alert">
16    <strong>Info!</strong> A new version of the app is available.
17  </div>
18  <h2 class="h4 mb-3 mt-4">2. Dismissible Alert (needs the JS bundle)</h2>
19  <!-- alert-dismissible -> adds right padding for the close button
20  <!-- fade show -> animated fade-out on close
21  <!-- data-bs-dismiss="alert" -> Bootstrap JS listens for this attribute
22  <!-- and removes the element. No JS written by us. -->
23  <div class="alert alert-warning alert-dismissible fade show" role="alert">
24    <strong>Heads up!</strong> Your session will expire in 5 minutes.
25    <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
26  </div>
27  <h2 class="h4 mb-3 mt-4">3. Badges</h2>
28  <!-- A badge is a small pill label. position-relative/absolute is NOT needed
29  <!-- here because the badge sits inline inside the button. -->
30  <button type="button" class="btn btn-primary">
31    Notifications <span class="badge bg-danger">3</span>
32  </button>
33  <span class="badge bg-secondary ms-2">New</span>
34  <span class="badge rounded-pill bg-success ms-2">Active</span>
35  <h2 class="h4 mb-3 mt-4">4. Three-Level Breadcrumb</h2>
36  <!-- aria-label tells assistive tech this nav is a breadcrumb trail.
37  <!-- The last item gets 'active + aria-current="page"' and is NOT a link. -->
38  <nav aria-label="Breadcrumb">
39    <ol class="breadcrumb">
40      <li class="breadcrumb-item"><a href="#">Home</a></li>
41      <li class="breadcrumb-item"><a href="#">Departments</a></li>
42      <li class="breadcrumb-item active" aria-current="page">Computer Science</li>
43    </ol>
44  </nav>
45  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>

```

1. Static Alerts (four contextual types)

Success! Your order has been placed.

Error! Payment failed. Please try again.

Warning! Your subscription expires in 3 days.

Info: A new version of the app is available.

2. Dismissible Alert (needs the JS bundle)

Heads up! Your session will expire in 5 minutes.

3. Badges

Notifications 3 New Active

4. Three-Level Breadcrumb

[Home](#) / [Departments](#) / [Computer Science](#)

Task 5 — Buttons & Button Groups

5.1 Concept

Buttons are the single most-used interactive element on the web, and Bootstrap standardizes their appearance and states with one base class (.btn) combined with a contextual color class (.btn-primary, .btn-outline-secondary, etc.), a size modifier (.btn-lg, .btn-sm), and state modifiers (disabled, .active). Button groups (.btn-group) visually cluster related actions — such as text alignment controls in a rich-text editor — into one seamless-looking control instead of several separate, awkwardly spaced buttons.

Class	Meaning / Typical Use
btn-primary	The main call-to-action color of the site theme (usually blue)
btn-secondary	A neutral, lower-emphasis action
btn-success / btn-danger	Confirm / destructive actions (Save vs Delete)
btn-outline-*	A lower-emphasis variant with a colored border and transparent background
btn-lg / btn-sm	Larger or smaller button sizing
disabled / aria-disabled	Visually and functionally disables the button

Real-World Anchor: A typical SaaS pricing page uses btn-primary btn-lg for the main 'Start Free Trial' button (highest visual weight) and btn-outline-secondary for a secondary 'Contact Sales' option right next to it — the visual hierarchy is created entirely through Bootstrap's button variant classes, guiding most users toward the primary action.

```

<button class="btn btn-primary btn-lg">Start Free Trial</button>
<button class="btn btn-outline-secondary btn-lg">Contact Sales</button>

<!-- Button Group -->
<div class="btn-group" role="group" aria-label="Text alignment">
  <button type="button" class="btn btn-outline-dark">Left</button>
  <button type="button" class="btn btn-outline-dark">Center</button>

```

```

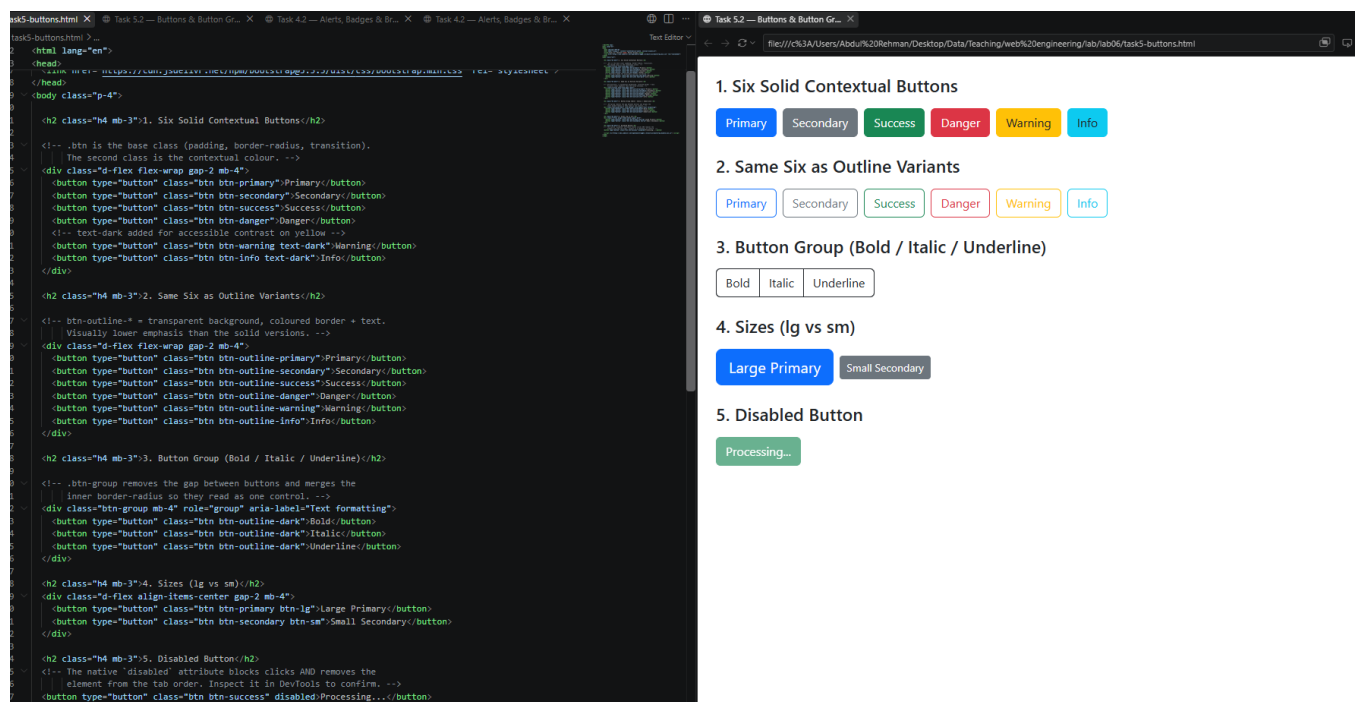
<button type="button" class="btn btn-outline-dark">Right</button>
</div>

<!-- Disabled state -->
<button class="btn btn-success" disabled>Processing...</button>

```

5.2 Lab Task

- Create a row of six buttons demonstrating all six contextual colors (primary, secondary, success, danger, warning, info).
- Recreate the same six buttons using the outline-* variants and compare the visual difference.
- Build a three-button toolbar using .btn-group representing 'Bold / Italic / Underline'.
- Add one large (btn-lg) primary button and one small (btn-sm) secondary button side by side and note the size difference.
- Add a disabled button and confirm in DevTools that the disabled attribute is present and the button cannot be clicked.



Task 6 — Cards, Media Object & Carousel

6.1 Concept

A card (.card) is Bootstrap's flexible, boxed content container — an image, a heading, body text, and action buttons grouped inside a bordered box with consistent padding. Because a card is a single reusable unit, it is the backbone of almost every modern grid of products, blog posts, or team-member profiles you see on the web. The media object pattern places an image or icon beside a block of text (e.g. an avatar next to a comment) using Flexbox alignment classes. A carousel (.carousel) rotates through a set of images or slides automatically or via manual controls — commonly used for hero banners and image galleries.

Real-World Anchor: Netflix-style content rows, Amazon product grids, and blog homepages (e.g. Medium, Dev.to) are all, structurally, a responsive grid of cards: an image on top, a title, a short excerpt, and a 'Read more' or 'Add to cart' button — exactly the card anatomy you will build in this task.

```

<div class="card" style="width: 18rem;">
  
  <div class="card-body">
    <h5 class="card-title">Wireless Headphones</h5>
    <p class="card-text">Over-ear, noise-cancelling, 30-hour battery life.</p>
    <a href="#" class="btn btn-primary">Add to Cart</a>
  </div>
</div>

<!-- Media Object -->
<div class="d-flex">
  
  <div>
    <h6 class="mb-1">Ayesha Khan</h6>
    <p class="mb-0">Great product, fast delivery!</p>
  </div>
</div>

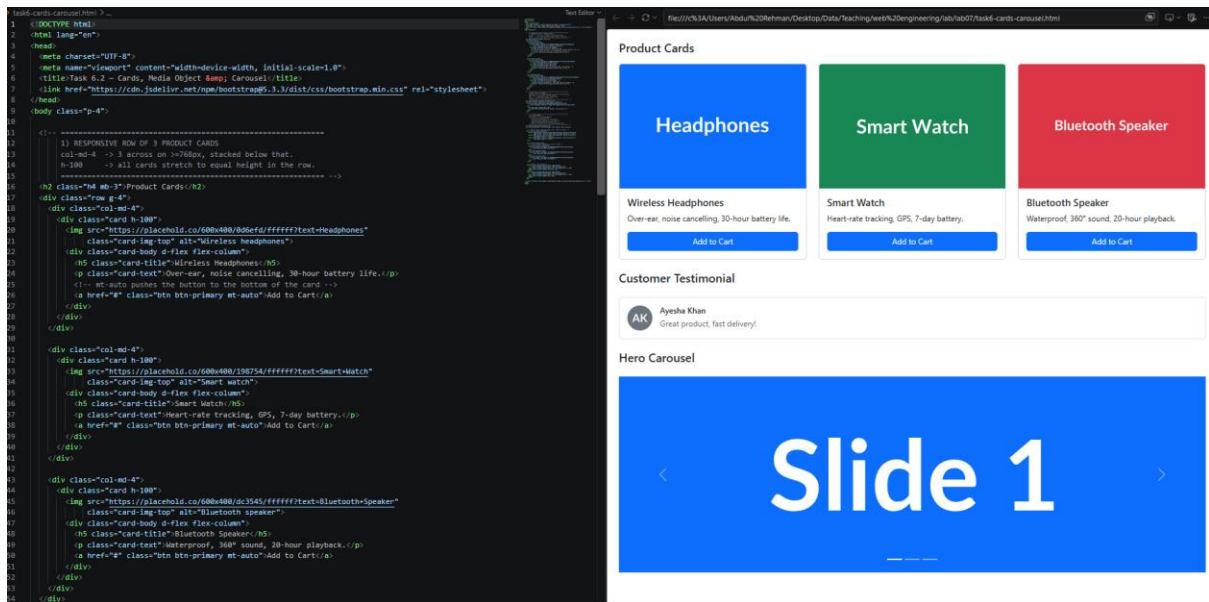
<!-- Carousel (abbreviated) -->
<div id="heroCarousel" class="carousel slide" data-bs-ride="carousel">
  <div class="carousel-inner">
    <div class="carousel-item active"></div>
    <div class="carousel-item"></div>
  </div>
  <button class="carousel-control-prev" type="button" data-bs-
  target="#heroCarousel" data-bs-slide="prev">
    <span class="carousel-control-prev-icon"></span>
  </button>
  <button class="carousel-control-next" type="button" data-bs-
  target="#heroCarousel" data-bs-slide="next">
    <span class="carousel-control-next-icon"></span>
  </button>
</div>

```

The `data-bs-ride="carousel"` attribute tells Bootstrap's JS to auto-advance the slides on page load; `data-bs-target` links the prev/next buttons to the specific carousel by its id. Exactly one `.carousel-item` must carry the active class, or the carousel will render as an empty box.

6.2 Lab Task

- Build a responsive row of 3 product cards (image placeholder, title, short description, and an 'Add to Cart' button), using `col-md-4` so they sit 3-across on desktop and stack on phones.
- Add a media-object-style testimonial block below the cards with a circular avatar placeholder beside a name and short quote.
- Build a 3-slide image carousel with previous/next controls and confirm both manual navigation and auto-advance work.
- Add carousel indicators (the small dots at the bottom of the carousel) linking to each slide.



Task 7 — Collapse & Accordion

7.1 Concept

Collapse is a Bootstrap JavaScript behavior that toggles the visibility of content by sliding it open or closed, without a full page reload. An accordion is simply a set of collapse panels wired together so that, typically, opening one panel closes the others — useful for showing a lot of content (FAQs, filter options) in a small vertical space. Both rely on the same `data-bs-toggle="collapse"` mechanism used elsewhere in Bootstrap's JS-driven components.

Real-World Anchor: FAQ pages on almost every SaaS and e-commerce site (Daraz's product Q&A section, most software pricing pages) use an accordion so dozens of questions can live on one page without overwhelming the user — only the question being read is expanded at any time.

```
<div class="accordion" id="faqAccordion">
  <div class="accordion-item">
    <h2 class="accordion-header">
      <button class="accordion-button" type="button" data-bs-toggle="collapse"
        data-bs-target="#faq1">
        What is your return policy?
      </button>
    </h2>
    <div id="faq1" class="accordion-collapse collapse show" data-bs-
      parent="#faqAccordion">
      <div class="accordion-body">Items can be returned within 14 days of
        delivery.</div>
    </div>
  </div>
  <div class="accordion-item">
    <h2 class="accordion-header">
      <button class="accordion-button collapsed" type="button" data-bs-
        toggle="collapse" data-bs-target="#faq2">
        Do you ship internationally?
      </button>
    </h2>
    <div id="faq2" class="accordion-collapse collapse" data-bs-
      parent="#faqAccordion">

```

```

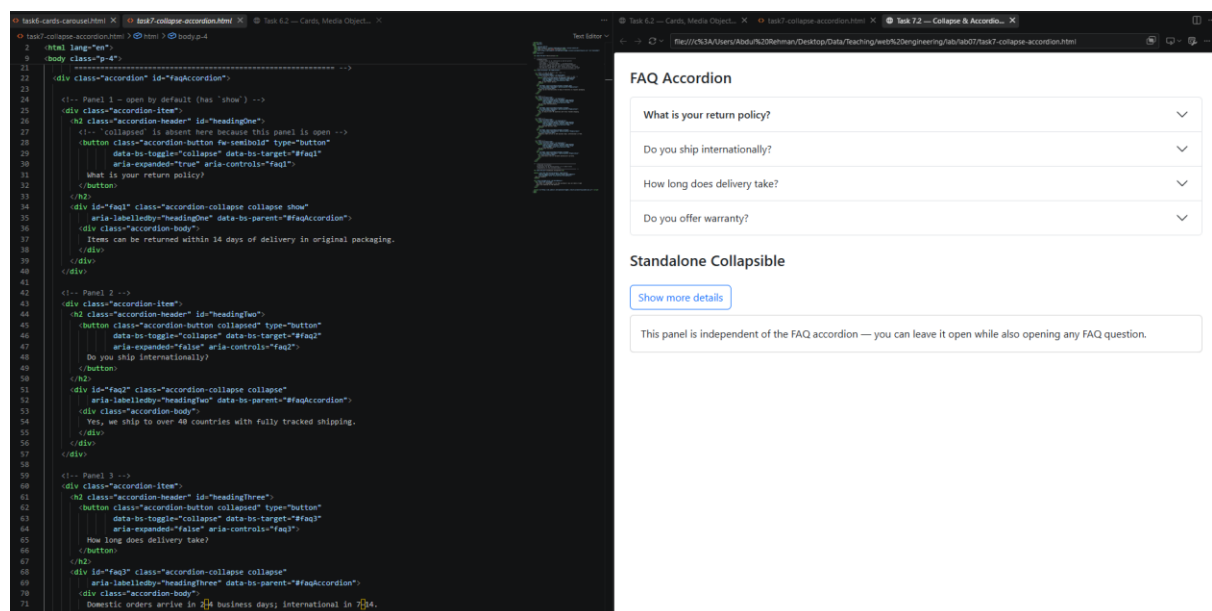
    <div class="accordion-body">Yes, to over 40 countries with tracked
shipping.</div>
  </div>
</div>
</div>

```

`data-bs-parent="#faqAccordion"` is what makes the panels mutually exclusive: opening one tells Bootstrap's JS to close any other open panel that shares the same parent. Removing that attribute turns the same markup into independent collapsibles that can all be open at once.

7.2 Lab Task

- Build a 4-question FAQ accordion where opening one question closes any other open question.
- Add a standalone collapsible 'Show more details' section elsewhere on the page (not part of the accordion) using a plain `<button data-bs-toggle="collapse">`, and confirm it can stay open independently of the accordion state.
- Style the accordion header button using a Bootstrap background color utility so the active question stands out.



Task 8 — Forms, Custom Forms & Input Groups

8.1 Concept

Raw HTML form controls render differently across browsers and operating systems. Bootstrap's form classes (`.form-control`, `.form-select`, `.form-check`) normalize their appearance, spacing, and focus states so a text field, dropdown, checkbox, and radio button all look like they belong to the same design system. 'Custom forms' extend this further by re-skinning checkboxes, radios, range sliders, and switches with fully custom, theme-aware graphics instead of the browser's native controls. An input group (`.input-group`) visually attaches an icon, label, or button to the side of an input — like a currency symbol before a price field or a 'Search' button glued to the right edge of a search box.

Real-World Anchor: A booking site's search bar (e.g. an airline or hotel search) commonly uses an input group with a calendar icon docked to the left of the date field and a 'Search' button docked to the right of the destination field, visually communicating that the icon and button belong to that specific input.

```
<form>
```

```

<div class="mb-3">
  <label for="email" class="form-label">Email address</label>
  <input type="email" class="form-control" id="email"
placeholder="name@example.com">
</div>

<div class="mb-3">
  <label for="country" class="form-label">Country</label>
  <select class="form-select" id="country">
    <option selected>Choose...</option>
    <option value="pk">Pakistan</option>
    <option value="us">United States</option>
  </select>
</div>

<div class="form-check mb-3">
  <input class="form-check-input" type="checkbox" id="terms">
  <label class="form-check-label" for="terms">I agree to the terms and
conditions</label>
</div>

<!-- Custom switch -->
<div class="form-check form-switch mb-3">
  <input class="form-check-input" type="checkbox" role="switch" id="notify">
  <label class="form-check-label" for="notify">Enable notifications</label>
</div>

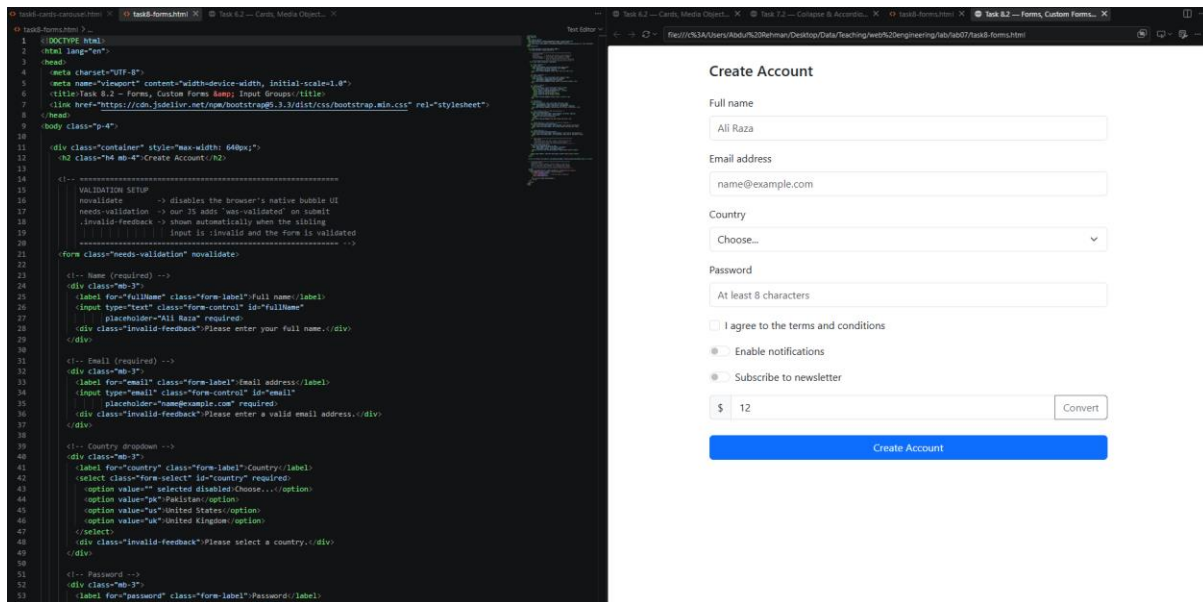
<!-- Input Group -->
<div class="input-group mb-3">
  <span class="input-group-text">$</span>
  <input type="text" class="form-control" placeholder="Amount">
  <button class="btn btn-outline-secondary" type="button">Convert</button>
</div>

  <button type="submit" class="btn btn-primary">Submit</button>
</form>

```

8.2 Lab Task

- Build a 'Create Account' form with a text input (name), email input, a select dropdown (country), a password input, and a checkbox for 'I agree to the terms'.
- Convert the checkbox into a custom form-switch and add one more switch for 'Subscribe to newsletter'.
- Add an input group combining a currency symbol, a numeric input, and a 'Convert' button as shown in the example.
- Add basic client-side validation using the required attribute on at least two fields, and use Bootstrap's .is-invalid / .invalid-feedback classes to show a custom error message when a required field is left empty.



Task 9 — List Group, Nav & Navbar

9.1 Concept

A list group (.list-group) displays a series of related, uniformly styled items — a to-do list, a settings menu, search results — as a clean vertical (or horizontal) list with built-in hover, active, and disabled states. A nav (.nav) is a more general-purpose set of navigation links that can be styled as plain links, pills (.nav-pills), or tabs (.nav-tabs). A navbar (.navbar) is Bootstrap's flagship responsive header component: on desktop it shows a horizontal row of links and a brand logo; on mobile, it automatically collapses the links behind a hamburger toggle button, because there usually is not enough horizontal space for them.

Real-World Anchor: Virtually every modern website header — from a university portal to a startup landing page — is a Bootstrap-style responsive navbar: a logo on the left, navigation links in the middle or right on desktop, and a hamburger icon that reveals a slide-down (or slide-in) menu on mobile. This single component is often the very first thing students recognize as 'Bootstrap' when they inspect a real site.

```

<!-- Responsive Navbar -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="#">MySite</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navMenu">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navMenu">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link active" href="#">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="#">About</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
      </ul>
    </div>
  </div>
</nav>

<!-- List Group -->
<ul class="list-group">
  <li class="list-group-item active">Inbox</li>

```

```

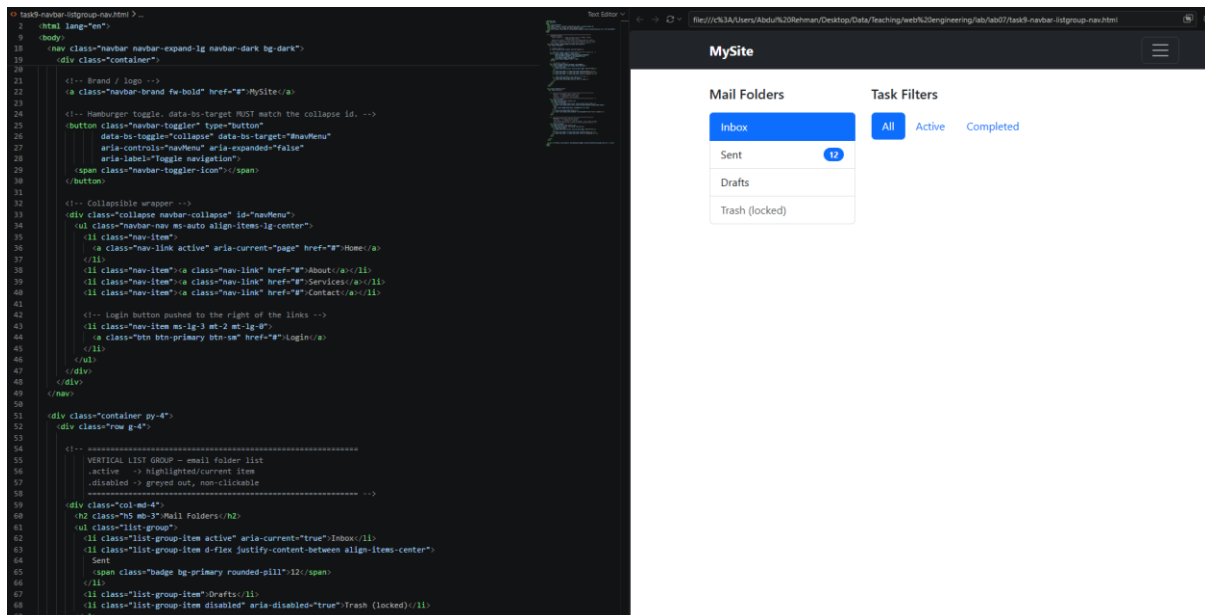
<li class="list-group-item">Sent</li>
<li class="list-group-item disabled">Drafts (locked)</li>
</ul>

```

navbar-expand-lg means the navbar stays expanded (links visible, no hamburger) from the lg breakpoint upward, and collapses into the hamburger menu below it — you can substitute -sm, -md, or -xl to change exactly where the collapse happens based on how many links you have and how much horizontal space they need.

9.2 Lab Task

- Build a responsive navbar with a brand name, 4 nav links, and a right-aligned 'Login' button (use ms-auto to push items to the right).
- Resize the browser below the chosen breakpoint and confirm the links collapse behind a working hamburger toggle.
- Build a vertical list group representing an email client's folder list (Inbox, Sent, Drafts, Trash), marking one item active and one item disabled.
- Build a horizontal pill-style nav (nav nav-pills) representing tab-like filters (e.g. 'All / Active / Completed' for a to-do app) and mark one pill active.



Task 10 — Modal, Toasts, Popovers & Tooltips

10.1 Concept

These four components all display extra information on top of the current page, but for different purposes and durations. A modal is a large dialog box that interrupts the user and requires a decision (confirm a delete, complete a login) — the rest of the page is dimmed and un-clickable behind it. A toast is a small, temporary, self-dismissing notification in a corner of the screen (e.g. 'Message sent') that does not block interaction with the rest of the page. A tooltip is a tiny text hint that appears when hovering or focusing on an element, briefly explaining what it does. A popover is like a bigger tooltip — it can contain a heading and a longer body of content, usually triggered by a click rather than a hover.

Component	Blocks the page?	Trigger	Typical Use
Modal	Yes	Click (button)	Confirm delete, login form, image lightbox
Toast	No	Programmatic / after an action	"Saved successfully", "New message received"
Tooltip	No	Hover / focus	Brief label for an icon-only button
Popover	No	Click (usually)	Short help text with a heading and body

Real-World Anchor: Gmail shows a toast at the bottom-left ('Message sent Undo') after sending an email, uses a modal to confirm permanently deleting an email from Trash, and uses tooltips on its icon-only toolbar buttons (archive, delete, mark unread) so first-time users know what each icon does without cluttering the toolbar with text labels.

```

<!-- Modal -->
<button class="btn btn-danger" data-bs-toggle="modal" data-bs-target="#confirmDelete">Delete Item</button>

<div class="modal fade" id="confirmDelete" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">Confirm Deletion</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">Are you sure you want to delete this item? This cannot be undone.</div>
      <div class="modal-footer">
        <button class="btn btn-secondary" data-bs-dismiss="modal">Cancel</button>
        <button class="btn btn-danger">Delete</button>
      </div>
    </div>
  </div>
</div>

<!-- Toast -->
<div class="toast" role="alert">
  <div class="toast-header"><strong class="me-auto">Notification</strong></div>
  <div class="toast-body">Your changes have been saved.</div>
</div>

<!-- Tooltip (requires JS initialization) -->
<button class="btn btn-outline-dark" data-bs-toggle="tooltip" title="Archive this email">📁</button>

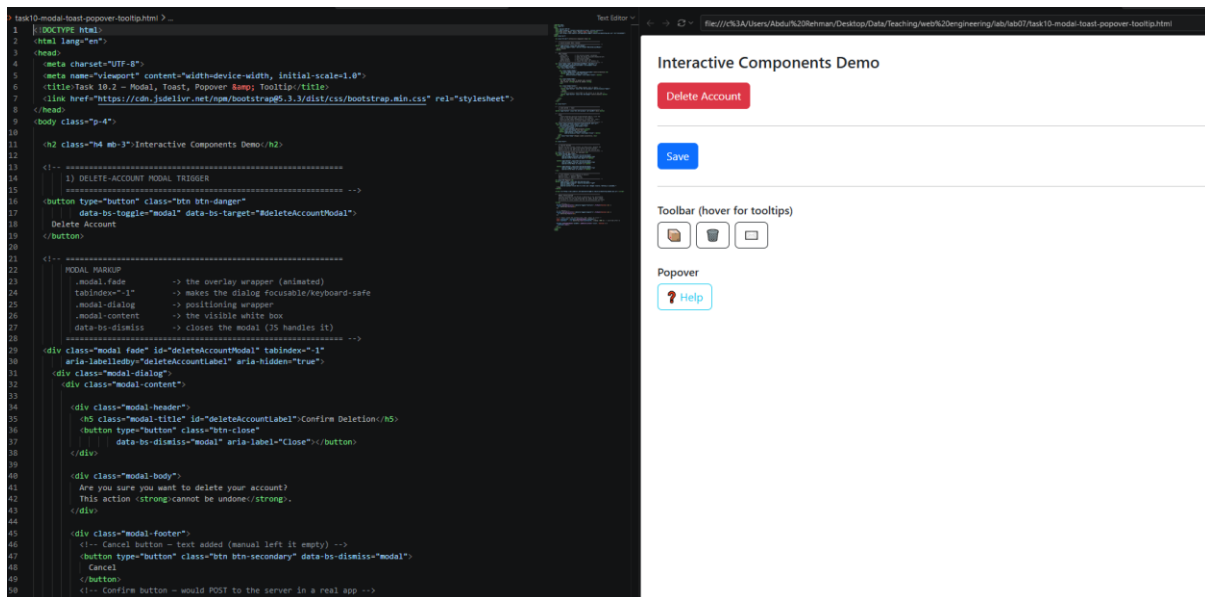
<script>
  // Tooltips must be manually activated in Bootstrap 5
  const tooltipEls = document.querySelectorAll('[data-bs-toggle="tooltip"]');
  tooltipEls.forEach(el => new bootstrap.Tooltip(el));
</script>

```

Unlike modals, dropdowns, and collapses, tooltips and popovers are opt-in for performance reasons: Bootstrap does not scan the whole page for them automatically, so you must explicitly create a new `bootstrap.Tooltip(element)` (or `Popover`) instance in a small script for each one, exactly as shown above.

10.2 Lab Task

- Build a 'Delete Account' button that opens a confirmation modal with Cancel and Confirm Delete buttons.
- Build a toast that appears (triggered by a 'Save' button) confirming 'Changes saved successfully', and let it auto-hide after a few seconds.
- Add tooltips to at least three icon-only buttons explaining their function, and initialize them with the required JavaScript snippet.
- Add a popover triggered by clicking a 'Help' icon, containing a short heading and one sentence of body text.



Task 11 — Paginators, Progress Bars & Spinners

11.1 Concept

A paginator (.pagination) breaks a long list of results into numbered pages so the user is not overwhelmed with hundreds of items at once and the server does not need to send them all at once. A progress bar (.progress) visually communicates how much of a task is complete — a file upload, a multi-step form, a course completion percentage. A spinner (.spinner-border / .spinner-grow) indicates that something is loading when the wait time is unknown or too short to show an exact percentage.

Real-World Anchor: A search engine results page uses numbered pagination at the bottom ('1 2 3 ... Next'); a cloud storage upload widget shows a progress bar filling as a file uploads; and almost any app shows a small spinner in place of a button's text (e.g. 'Log In' becomes a spinning circle) the instant it is clicked, giving the user immediate confirmation that their click registered.

```
<!-- Pagination -->
<nav aria-label="Page navigation">
  <ul class="pagination">
    <li class="page-item disabled"><a class="page-link" href="#">Previous</a></li>
    <li class="page-item active"><a class="page-link" href="#">1</a></li>
    <li class="page-item"><a class="page-link" href="#">2</a></li>
    <li class="page-item"><a class="page-link" href="#">3</a></li>
    <li class="page-item"><a class="page-link" href="#">Next</a></li>
  </ul>
</nav>
```

```

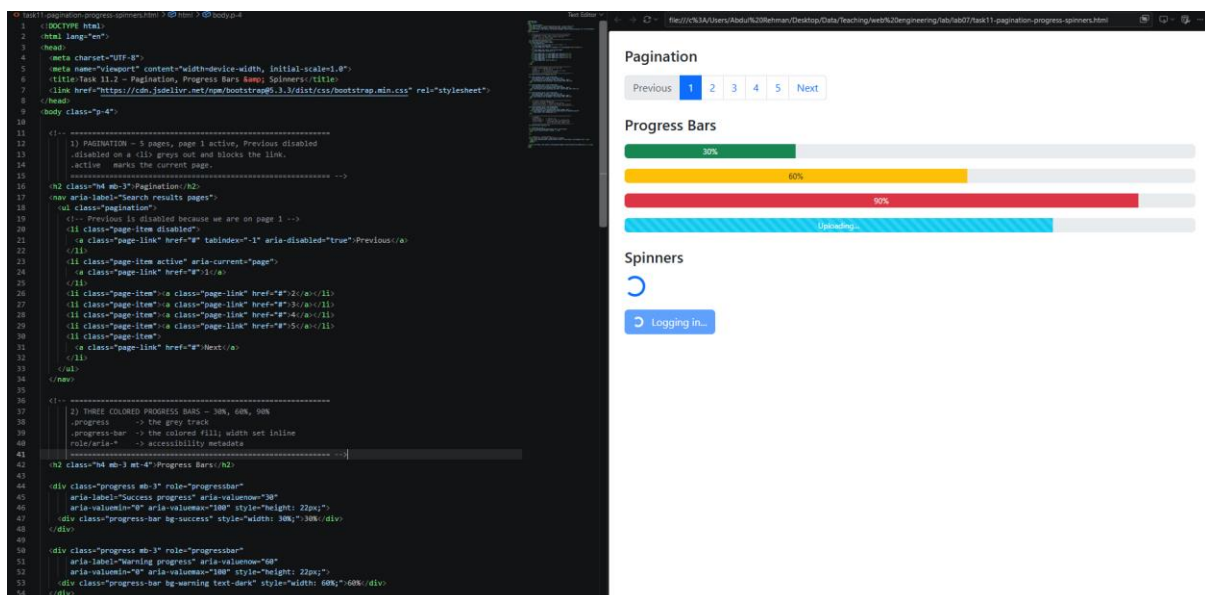
<!-- Progress Bar -->
<div class="progress" role="progressbar" aria-valuenow="65" aria-valuemin="0" aria-
valuemax="100">
  <div class="progress-bar bg-success" style="width: 65%">65%</div>
</div>

<!-- Spinners -->
<div class="spinner-border text-primary" role="status">
  <span class="visually-hidden">Loading...</span>
</div>
<button class="btn btn-primary" disabled>
  <span class="spinner-border spinner-border-sm" role="status"></span>
  Logging in...
</button>

```

11.2 Lab Task

- Build a 5-page pagination control with the current page marked active and the Previous button disabled (since page 1 is active).
- Build 3 progress bars in different colors (bg-success, bg-warning, bg-danger) representing 30%, 60%, and 90% completion respectively.
- Add a striped and animated progress bar using progress-bar-striped progress-bar-animated.
- Recreate the 'Logging in...' button pattern shown above using a small spinner inside a disabled button.



Task 12 — Tables

12.1 Concept

Plain HTML `<table>` elements are functional but visually cramped and hard to scan. Bootstrap's `.table` class adds sensible padding and borders, and modifier classes let you add zebra-stripping (`.table-striped`) for easier row-scanning, hover highlighting (`.table-hover`) to help the eye track a row across many columns, and contextual row/cell coloring (`.table-success`, `.table-danger`) to flag specific data (e.g. a failed transaction row). Because tables do not reflow like text, wrapping a table in `.table-responsive` adds horizontal scrolling on narrow screens instead of squeezing every column illegibly.

Real-World Anchor: An admin dashboard's 'Orders' page or a bank's transaction history table typically combines striping, hover highlighting, and a contextual color on rows that failed or are pending — exactly the pattern in this task — so an operator can scan hundreds of rows quickly and immediately spot the ones that need attention.

```
<div class="table-responsive">
  <table class="table table-striped table-hover">
    <thead>
      <tr><th>#</th><th>Customer</th><th>Amount</th><th>Status</th></tr>
    </thead>
    <tbody>
      <tr><td>1</td><td>Ali Raza</td><td>$120</td><td>Completed</td></tr>
      <tr class="table-warning"><td>2</td><td>Sara
      Khan</td><td>$85</td><td>Pending</td></tr>
      <tr class="table-danger"><td>3</td><td>Bilal
      Ahmed</td><td>$40</td><td>Failed</td></tr>
    </tbody>
  </table>
</div>
```

12.2 Lab Task

- Build a table of at least 6 rows representing a simple order list (Order ID, Customer Name, Amount, Status).
- Apply table-striped and table-hover and confirm both effects are visible when moving the mouse over rows.
- Color-code rows using table-success, table-warning, and table-danger based on the order status column.
- Wrap the table in table-responsive, then shrink the browser to a narrow width to confirm the table scrolls horizontally instead of breaking its layout.

The screenshot shows a web browser window with a table titled "Order List". The table has 5 columns: #, Order ID, Customer, Amount, and Status. There are 6 rows of data. The rows are color-coded based on their status: Completed (green), Pending (yellow), and Failed (red). The table is wrapped in a "table-responsive" container, which is visible as a dark background on the left side of the browser window.

#	Order ID	Customer	Amount	Status
1	#ORD-1001	Ali Raza	\$120	Completed
2	#ORD-1002	Hina Malik	\$75	Completed
3	#ORD-1003	Sara Khan	\$85	Pending
4	#ORD-1004	Bilal Ahmed	\$40	Failed
5	#ORD-1005	Usman Tariq	\$210	Pending
6	#ORD-1006	Zainab Fatima	\$60	Completed

Task 13 — Styling Essentials: Floats, Flex, Position, Borders & Shadows, Visibility

13.1 Concept

Beyond components, Bootstrap ships a full set of low-level layout utility classes that replace hand-written CSS for the most common layout tasks. Float utilities (`.float-start` / `.float-end`) wrap text around an element the old way. Flex utilities (`.d-flex`, `.justify-content-*`, `.align-items-*`) expose the CSS Flexbox model as classes, letting you build custom row/column layouts, centering, and spacing without leaving your HTML file. Position utilities (`.position-relative`, `.position-absolute`, `.top-0`, `.end-0`, `.translate-middle`) place an element precisely, commonly used for badges overlapping the corner of an icon or images. Border and shadow utilities (`.border`, `.border-primary`, `.rounded`, `.shadow`, `.shadow-lg`) add quick visual polish. Visibility utilities (`.visible` / `.invisible`, alongside the display utilities from Task 3) control whether an element is seen without necessarily removing it from the document flow.

Category	Example Classes	Effect
Flex container	<code>d-flex</code> , <code>flex-row</code> , <code>flex-column</code>	Turns an element into a flex container, row or column direction
Flex alignment	<code>justify-content-between</code> , <code>align-items-center</code>	Distributes/aligns flex children along the main/cross axis
Position	<code>position-absolute</code> , <code>top-0</code> , <code>end-0</code> , <code>translate-middle</code>	Precisely places an element relative to its positioned parent
Borders	<code>border</code> , <code>border-2</code> , <code>border-primary</code> , <code>rounded-circle</code>	Adds/colors/rounds an element's border
Shadows	<code>shadow-sm</code> , <code>shadow</code> , <code>shadow-lg</code>	Adds a soft drop shadow of increasing intensity
Visibility	<code>visible</code> , <code>invisible</code>	Shows/hides an element while it still occupies layout space

Real-World Anchor: A shopping app's cart icon with a small red badge sitting exactly on its top-right corner (e.g. Daraz, Amazon mobile apps) is built with `position-relative` on the icon and `position-absolute top-0 start-100 translate-middle` on the badge — a combination so common it appears almost verbatim in Bootstrap's own official documentation examples.

```
<!-- Flexbox: evenly spaced toolbar -->
<div class="d-flex justify-content-between align-items-center p-3 bg-light">
  <span class="fw-bold">Dashboard</span>
  <button class="btn btn-sm btn-primary">Logout</button>
</div>

<!-- Position: badge on a corner icon -->
<button class="btn btn-outline-dark position-relative">
  Cart
  <span class="position-absolute top-0 start-100 translate-middle badge rounded-pill bg-danger">
    3
  </span>
</button>

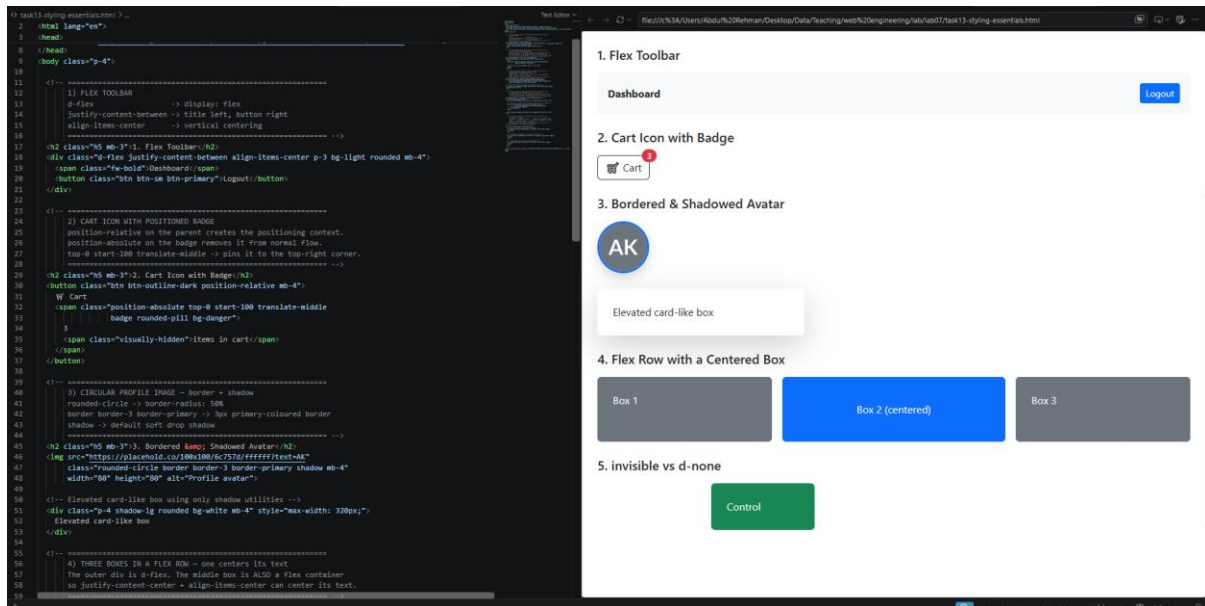
<!-- Borders + Shadow -->

<div class="p-4 shadow-lg rounded">Elevated card-like box</div>
```

13.2 Lab Task

- Build a toolbar using `d-flex justify-content-between` with a title on the left and a button on the right.

- Recreate the cart-icon-with-badge pattern shown above using position-relative and position-absolute.
- Style a circular profile image with a colored border and a shadow using rounded-circle, border, border-primary, and shadow.
- Build 3 boxes side by side using d-flex, and use justify-content-center and align-items-center to perfectly center text inside one of them.
- Demonstrate the difference between invisible and d-none on two identical boxes: apply invisible to one and d-none to the other, and use DevTools to show that the invisible box still occupies space while the d-none box does not.



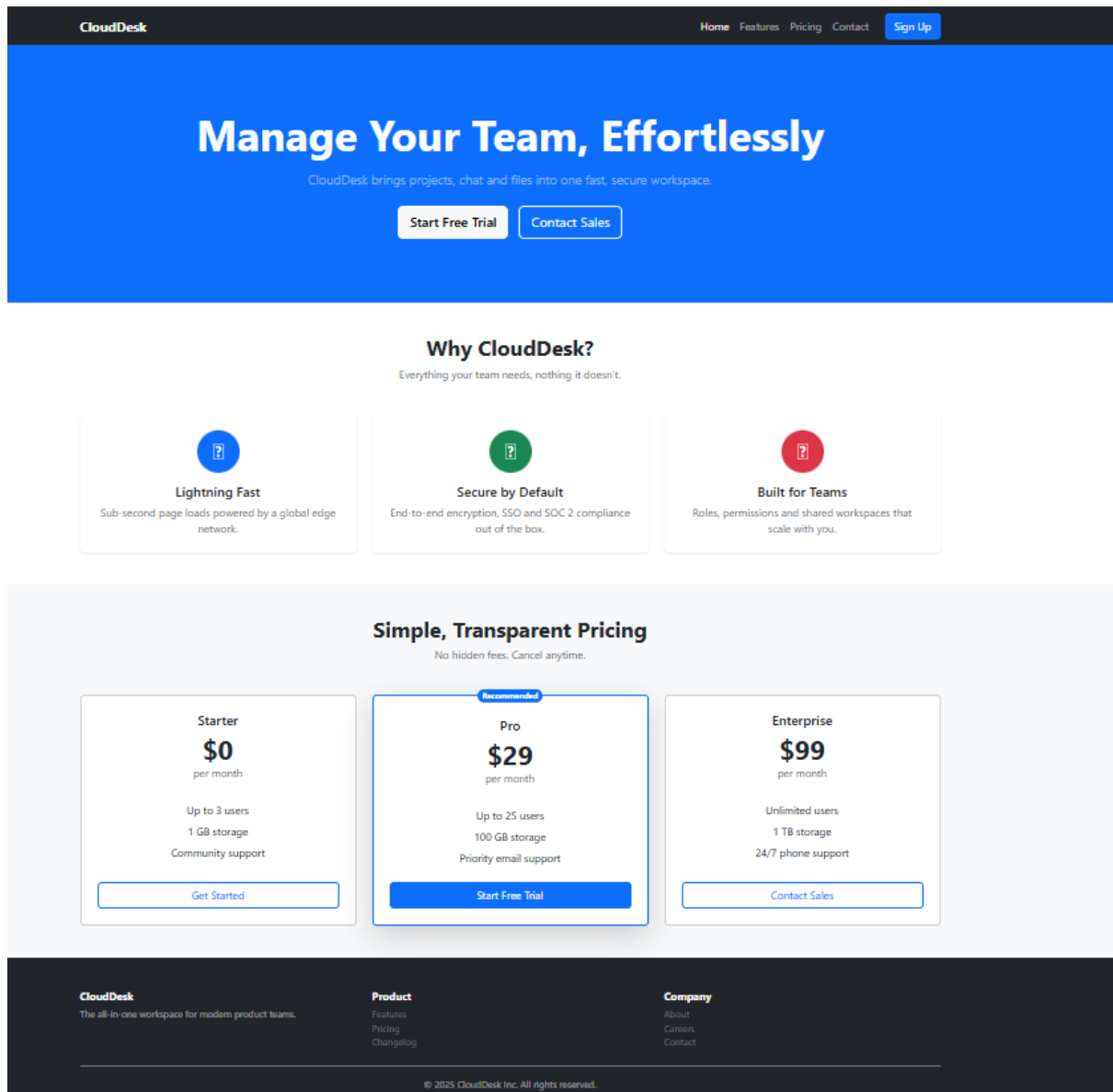
Practice Exercise (To Be Submitted)

Using everything covered in Tasks 1–13, choose ONE of the two reference designs below and rebuild it as closely as possible using only Bootstrap 5 classes (no custom CSS beyond minor image placeholder styling). These reference designs are provided so you practice reading a real interface and mapping what you see back to the correct Bootstrap components — a skill you will use constantly in real front-end work.

Option A — SaaS Landing Page

Recreate a typical SaaS product landing page containing:

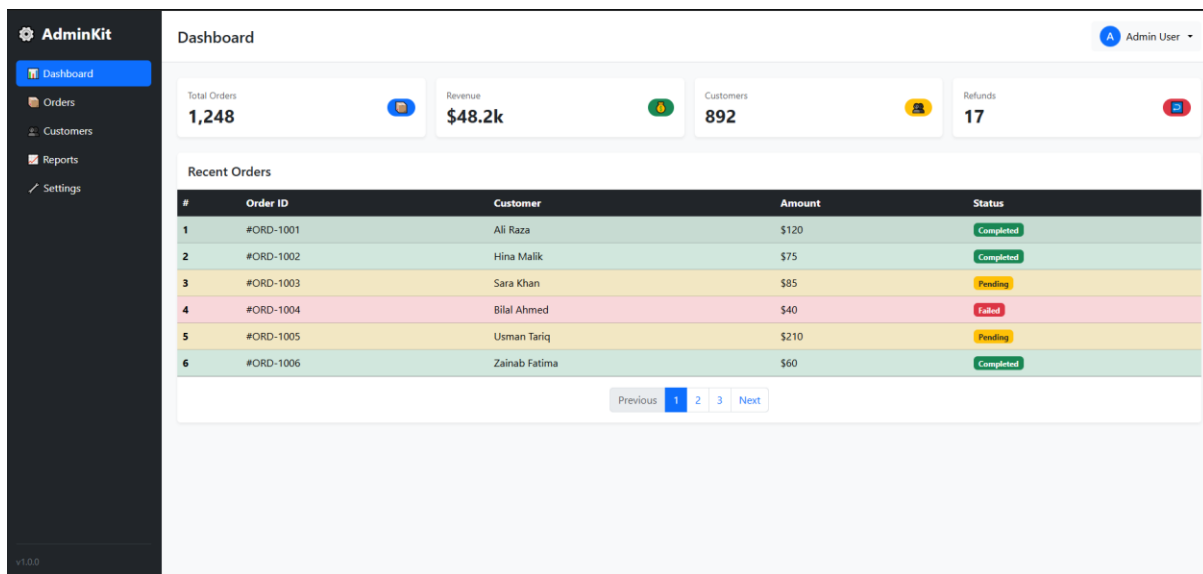
- A responsive navbar with a brand name, 3–4 nav links, and a 'Sign Up' button, collapsing into a hamburger menu on mobile.
- A full-width hero section (container-fluid) with a large bold centered heading, a muted subtitle, and two call-to-action buttons (btn-primary and btn-outline-secondary).
- A 3-column features section using cards, each with an icon/image placeholder, a heading, and a short description.
- A pricing section with 3 cards of different emphasis (use border-primary and shadow-lg on the 'recommended' plan to make it stand out).
- A footer with 3 columns of links and a bottom row with copyright text, built with the grid system.



Option B — Admin Dashboard Page

Recreate a typical admin dashboard page containing:

- A dark sidebar navigation (nav flex-column) with icons and labels for at least 5 sections, with one item marked active.
- A top bar using d-flex justify-content-between showing a page title on the left and a user-profile dropdown on the right.
- A row of 4 'stat cards' (col-md-3 each) showing a KPI number, a label, and a colored icon/badge.
- A striped, hover-enabled, responsive data table listing at least 6 rows of records with a status column color-coded using table-success / table-warning / table-danger.
- A pagination control beneath the table.



For whichever option you choose, your final HTML file must use only Bootstrap's grid, utility, and component classes — do not hand-write layout CSS. Where the reference design uses a real photo, use a plain gray placeholder box or an image from <https://placeholder.co> instead.

[SCREENSHOT: Your own completed page rendered in Chrome at desktop width, next to the reference screenshot, for visual side-by-side comparison]

[SCREENSHOT: Your own completed page rendered in Chrome's mobile device toolbar (e.g. iPhone 12 Pro width), showing the navbar collapsed and the grid columns stacked correctly]

Submission Instructions

This lab must be submitted as a single PDF file uploaded to the course eLearning portal before the announced deadline. Late submissions will be handled according to the course policy stated in the course outline.

Your submitted PDF must include, for every task and the Practice Exercise:

- The complete, working HTML source code for that task, formatted as plain, readable, monospaced text (a code screenshot from VS Code with visible syntax highlighting is acceptable).
- A corresponding screenshot of the actual rendered output in the browser, taken immediately after running the code — placed directly beside or beneath its matching code so the grader can verify code and output together.
- Answers to any in-task questions or predictions requested in the Lab Task sections (e.g. Task 4's dismiss confirmation, Task 13's invisible vs d-none comparison).
- Your answers to the Conceptual / Viva Questions section, written in your own words.

Do not submit a .zip of raw project files and do not submit a Word document — only a single PDF containing code and output together will be accepted. Name the file using the format RollNumber_LabWeek3_Bootstrap.pdf.

Conclusion

In this lab, students moved from hand-written CSS to Bootstrap, the industry's most widely adopted front-end framework. You learned how Bootstrap is structured and included in a project, how its mobile-first 12-column grid and breakpoint system drive responsive layout, and how to style content quickly using utility classes for typography, color, spacing, flexbox, position, borders, and shadows. You also implemented the full range of core UI components — alerts, badges, breadcrumbs, buttons, cards,

carousels, collapsibles, forms, list groups, navs, navbars, modals, toasts, tooltips, popovers, pagination, progress bars, spinners, and tables — and combined several of them into one complete, responsive page. These are the same building blocks used across the vast majority of production business websites and internal tools, and they form the practical foundation for the JavaScript-driven DOM manipulation and dynamic front-end work covered in the labs ahead.

End of Week 7 Lab Experiment